



LifeGoals

Aplicación multiplataforma para la gestión de metas inteligentes

Tutor:

Gabriel Pastor Martín

Participantes: Samuel Carro, Jose Angel, Israel Quintero, Victor Rufo

Índice

Índice	2
Equipo	5
1. Introducción	5
2. Objetivos del proyecto	6
2.1 Objetivo general	6
MVP — Producto mínimo viable (lo que debe funcionar para el TFG)	7
Fuera del MVP — trabajo futuro o descartado:	7
2.2 Objetivos específicos	7
Tabla global de tecnologías (resumen)	9
3. Firebase	10
3.1. Marco Teórico	10
3.1.1 Firebase	10
3.1.2 AI Logic	11
3.2. Aplicaciones similares (estado del arte)	11
3.3. Desarrollo del proyecto	12
3.3.1 Arquitectura del sistema	12
Visión general	12
Servicios utilizados	13
Metodología de desarrollo	14
Ventajas de esta arquitectura	14
3.3.2 Base de datos — Cloud Firestore	15
Datos iniciales — categorías	16
Reglas de seguridad (Firestore Security Rules)	16
Explicación actual de las reglas de Firestore	18
Decisiones de diseño	19
3.4. Lógica de datos e integración con IA	20
Cómo reemplaza Firebase al backend	20
Flujo de creación de una meta con IA	20
Prompt utilizado	21
Particularidades técnicas de la integración	21
Gestión de estado global — GoalsContext	22
Operaciones CRUD en Firestore	22
4. Frontend Android	23
4.1 Análisis del estado del arte y elección tecnológica	23

4.1.1 Comparativa Crítica de Frameworks de Desarrollo Móvil	23
4.2 Arquitectura Técnica y Patrones de Diseño de Software	24
4.2.1 Adaptación del Patrón MVVM a React	24
4.2.2 Gestión de Estado Global con Context API	25
4.3 Análisis Detallado de la Implementación por Ficheros (Lógica de Negocio)	26
4.3.1 El Corazón de la Lógica:	26
4.3.2 El Sistema de Rutas y Navegación: src/app/routes.jsx	27
4.4 Análisis Detallado de la Interfaz de Usuario (Capa de Presentación)	28
4.4.1 El Dashboard Central: src/app/pages/DashboardPage.jsx	28
4.4.2 Creación de Objetivos Asistida: src/app/pages/CreateGoalPage.jsx	29
4.4.3. Detalle, Gestión e Interacción:	30
4.5. Manual de Usuario Exhaustivo y Guía de Interacción Paso a Paso	30
4.5.1. Acceso Seguro y Gestión de su Cuenta Personal	30
4.5.2. Interpretar su Dashboard de Control Personal	32
4.5.3. El Proceso de Creación de una Meta con Ayuda de la IA	33
4.5.4. Gestión Diaria, Seguimiento y Cumplimiento	34
4.6. Ciclo de Vida del Software y Metodología de Desarrollo Aplicada	35
4.7. Guía Técnica Completa para el Despliegue e Instalación (Para Desarrolladores)	37
4.7.1. Prerrequisitos de Software del Sistema	37
4.7.2. Proceso de Instalación y Puesta en Marcha Paso a Paso	37
5. Frontend Web	39
5.1. Marco teórico y tecnologías utilizadas	39
5.1.1 React y arquitectura basada en componentes	39
5.1.2 Vite como herramienta de desarrollo	39
5.1.3 TypeScript, rutas y estilos	39
Tabla de tecnologías	40
5.2. Desarrollo del proyecto	41
5.2.1 Arquitectura general	41
5.2.2 Estructura de carpetas y archivos	41
5.2.3 Arranque de la aplicación	42
5.2.4 Sistema de rutas y protección	43
5.2.5 Autenticación de usuarios	44
5.2.6 Gestión de datos con Firestore	44
5.2.7 GoalsContext como núcleo de la aplicación	45
5.2.8 Integración de Gemini IA	46
5.2.9 Pantallas de la aplicación	47

5.2.10 Diseño UX/UI	48
5.2.11 Variables de entorno	49
6. Resultados y pruebas realizadas	50
Plan de pruebas funcionales	50
6.1 Resultados alcanzados	51
6.2 Capturas finales	51
7. Conclusiones	52
7.1 Conclusiones del proyecto	52
7.2 Trabajo futuro	52
8. Bibliografía	53
Anexos	54
Anexo A: Estructura Completa y Pormenorizada del Árbol de Directorios (FrontEnd Android)	54
Anexo B: Listado Detallado de Dependencias de Software Utilizadas (FrontEnd Android)	55
Anexo C: — Árbol del proyecto (FrontEnd Web)src/	56
Anexo D: — Interfaces principales (FrontEnd Web)	57
Anexo E: — Variables de entorno de ejemplo (FrontEnd Web)	57
Anexo F: Glosario de Conceptos Técnicos Utilizados en el Proyecto	58

Equipo

Tabla 1. Equipo de trabajo y reparto de secciones.

Persona	Rol	Secciones del TFG
José Ángel	Firebase Admin + Coordinación	3. Firebase
Víctor	Pruebas e integración	1. Introducción 2. Objetivos 6. Conclusiones 7. Pruebas e integración
Israel	Frontend Web (React + Vite)	5. FrontEnd Web
Samuel	Frontend Móvil (React Native)	4. FrontEnd Android

1. Introducción

El objetivo final de este proyecto es desarrollar una aplicación multiplataforma completa que funcione como un panel de control o *dashboard*, en el que un usuario pueda definir cuáles son sus metas a futuro, y la aplicación se encargará de desglosarlas en hitos inteligentes mediante la aplicación de inteligencia artificial, permitiendo también su seguimiento, modificación o eliminación.

Estas metas serán independientes entre usuarios y podrán clasificarse en cinco categorías distintas. Una vez creada una meta, una inteligencia artificial, tomando el rol de un coach de productividad, será la encargada de desarrollar cuáles son los siguientes pasos que debe tomar el usuario, ayudándolo a organizar mejor sus ideas y evitando así la frustración y el estrés de no saber cómo desarrollar sus propios objetivos y pasiones.

La idea surge a raíz de la curiosidad por buscar nuevas herramientas y tecnologías con las que no se hubiera experimentado hasta ahora, tales como la inteligencia artificial aplicada al desarrollo y comprensión de ideas proporcionadas por el usuario, o el uso de Firebase como base de datos en la nube, con el objetivo de comprobar cuáles son las ventajas que ofrece frente a una base de datos SQL convencional.

El frontend se ha desarrollado utilizando dos tecnologías principales: React y React Native. Debido a su naturaleza, su sintaxis y a la experiencia previa del equipo con ambas tecnologías, se consideró que eran las opciones más adecuadas para este proyecto.

La parte del servidor se desarrolló con Firebase, una plataforma de desarrollo en la nube que actúa como un hub en el que se pueden implementar fácilmente todos los servicios necesarios para la aplicación.

La aplicación está disponible en tres plataformas: web (React) y móvil/tablet (React Native para Android), ambas conectadas a la misma base de datos en la nube (Firebase), lo que permite al usuario acceder a sus metas desde cualquier dispositivo.

2. Objetivos del proyecto

2.1 Objetivo general

Desarrollar una aplicación funcional, lista para desplegar y que cumpla con un mínimo de cosas que deben funcionar, llamado el MVP (Minimum Viable Product).

MVP — Producto mínimo viable (lo que debe funcionar para el TFG)

- ~~Registro y login de usuario con email y contraseña (Firebase Auth)~~
- ~~Crear metas a largo plazo con categoría, fecha límite y descripción~~
- ~~Generación automática de hitos mediante IA (Gemini 2.5 Flash)~~
- ~~Marcar hitos como completados y ver el progreso visual~~
- ~~Dashboard personal con resumen de metas activas~~
- ~~Aplicación web funcional (React + Vite)~~
- ~~Aplicación móvil Android funcional (React Native)~~
- ~~Login con Firebase Authenticator~~

Fuera del MVP — trabajo futuro o descartado:

- Notificaciones push en Android (recordatorios de hitos próximos)
- Notificaciones push en web
- Parte social / perfiles públicos
- Estadísticas avanzadas de productividad
- Conversación guiada con IA para refinar la meta antes de generar hitos
- Los 3 caminos alternativos para desarrollar una meta

2.2 Objetivos específicos

- Definir la arquitectura del sistema
 - Crear el proyecto Firebase y Firestore con datos iniciales
 - Implementar Firebase Authentication (registro y login)
 - Conectar la web (React) con Firebase
 - Conectar la app móvil (React Native) con Firebase
 - Implementar IA con Gemini 2.5 Flash (web y móvil)
 - Flujo completo funcionando: login → meta → hitos IA → progreso
 - Pruebas de integración completas (fase 1)
 - Pruebas de integración completas (fase 2)
 - Redactar documentación final y pasar a Word
-

Tabla global de tecnologías (resumen)

Tabla 2. *Tabla global de tecnologías utilizadas en el proyecto.*

Tecnología	Uso en el proyecto	Archivo principal
React	Pantallas y componentes	App.tsx, páginas
React Native	App Android	src/app/pages
TypeScript	Tipado de datos	GoalsContext.tsx
Vite	Build y entorno web	vite.config.ts
Firebase Auth	Registro, login, logout	firebase.ts
Cloud Firestore	Persistencia de datos	colecciones
Gemini API	Generación de hitos	gemini.ts / iaService.js
React Router	Navegación web	routes.tsx
React Navigation	Navegación móvil	AppNavigator.js
Tailwind CSS	Diseño web	src/styles

3. Firebase

3.1. Marco Teórico

3.1.1 Firebase

Firebase es una plataforma de desarrollo de aplicaciones web y móviles de Google que proporciona una suite de servicios en la nube, nos permite integrar la base de datos, la autenticación y la IA en un mismo lugar. En este proyecto se utilizan dos de sus servicios principales:

Firebase Authentication gestiona el registro e inicio de sesión de los usuarios mediante email y contraseña. Sustituye la implementación manual de JWT y Spring Security que se había planteado inicialmente, ofreciendo la misma funcionalidad con mayor seguridad y sin necesidad de un servidor propio. Cada usuario registrado recibe un UID único que se usa para identificar y filtrar sus metas en Firestore.

Cloud Firestore es la base de datos NoSQL en tiempo real donde se persisten todos los datos de la aplicación: metas, hitos y categorías.

Al ser una base de datos en la nube, tanto la aplicación web como la móvil acceden a los mismos datos en tiempo real sin necesidad de configurar ni mantener ningún servidor local.

Se optó por Firebase en lugar de PostgreSQL + Spring Boot por eliminar la dependencia de un servidor local, permitir acceso simultáneo desde web y móvil, y reducir significativamente la carga de desarrollo.

3.1.2 AI Logic

Gemini 2.5 Flash es el modelo de lenguaje grande (LLM) de Google utilizado en este proyecto para la generación automática de hitos. Se accede a él mediante la API REST de Google AI Studio, llamando al endpoint de generación de contenido con la clave de API obtenida gratuitamente en aistudio.google.com.

Se eligió Gemini 2.5 Flash por ser el modelo estable más reciente de Google en el momento del desarrollo, disponible de forma gratuita dentro de los límites del tier gratuito de Google AI Studio, y por ofrecer un excelente equilibrio entre velocidad y calidad de respuesta para la tarea de generación de hitos.

La llamada a la API se realiza directamente desde los clientes (web y móvil) mediante fetch a la URL: <https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent>

3.2. Aplicaciones similares (estado del arte)

Me+ — App de seguimiento de vida personal que permite definir grandes metas y descomponerlas. Es la referencia principal de diseño de LifeGoals, aunque nuestra propuesta añade generación automática de hitos con IA y se centra en metas a largo plazo.

Notion — Espacio de trabajo colaborativo. Se ha utilizado en este proyecto como herramienta interna de documentación antes del volcado final a Word.

Habitica — App de gestión de hábitos con gamificación. Orientada a hábitos diarios, no a metas a largo plazo.

3.3. Desarrollo del proyecto

3.3.1 Arquitectura del sistema

Visión general

La aplicación sigue una arquitectura **cliente-nube** donde los dos frontends (web y móvil) se comunican directamente con los servicios de Firebase y Google AI, eliminando la necesidad de un servidor backend propio.

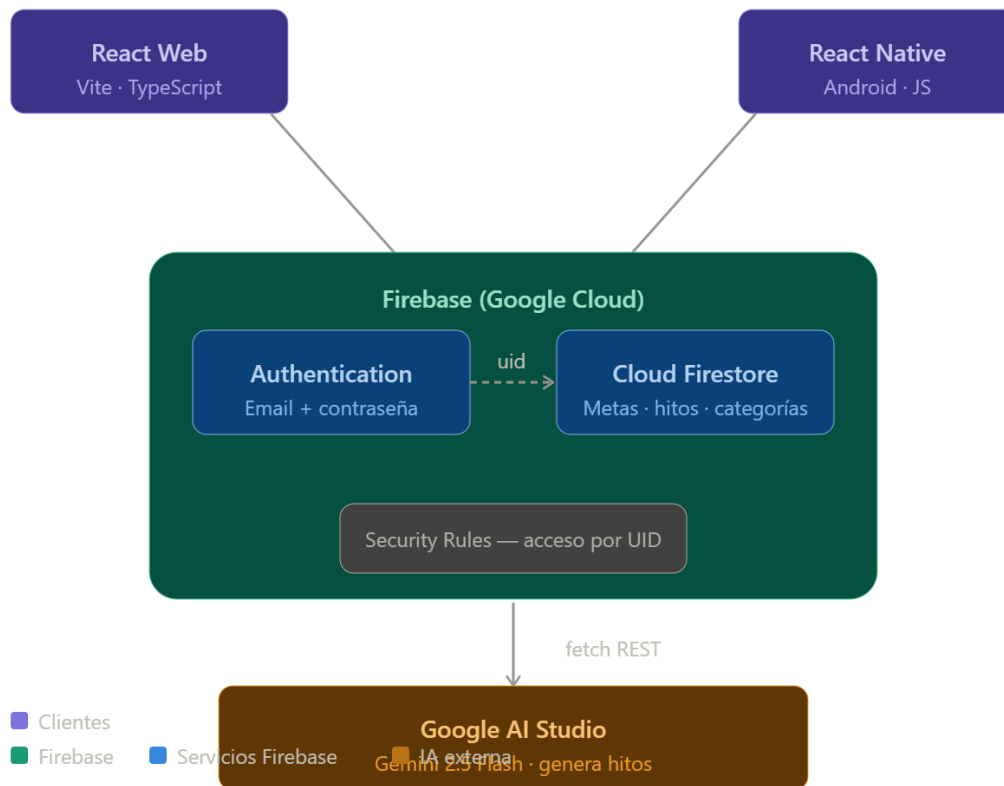


Figura 1. Diagrama de arquitectura del sistema. Elaboración propia.

Existen dos clientes independientes que acceden a los mismos datos:

- **React Web:** aplicación de escritorio accesible desde el navegador.
- **React Native Android:** aplicación móvil para Android.

Por qué se migró de Spring Boot a Firebase

El diseño inicial contemplaba un backend en Spring Boot con PostgreSQL.

Tras una valoración técnica del equipo, se decidió migrar a Firebase por los siguientes motivos:

- Eliminación de la dependencia de un servidor local para desarrollar y demostrar la app.
- Acceso simultáneo desde web y móvil sin configuración de red.
- Gestión de autenticación y seguridad integradas y gratuitas.
- Reducción significativa de la carga de desarrollo al eliminar una capa completa del sistema.
- Simplificación general a la hora de exponer el proyecto

Servicios utilizados

Firebase Authentication gestiona el ciclo de vida de los usuarios. Cada usuario recibe un UID único que se usa para asociar sus metas en Firestore y garantizar que solo él puede acceder a ellas.

Cloud Firestore almacena todos los datos en colecciones, es el que sustituye PostgreSQL. Las metas están filtradas por el UID del usuario, garantizando aislamiento total entre usuarios.

AI Logic (Google AI Studio) permite llamar al modelo de IA directamente desde los clientes mediante fetch a la API REST. Se usa para generar automáticamente los hitos de cada meta, y en caso de error al generar los hitos inteligentes, se generan unos hitos por defecto para que el usuario tenga al menos una base.

Metodología de desarrollo

El desarrollo se ejecutó siguiendo una metodología ágil e iterativa dividida en cuatro fases:

1. Análisis y definición de requerimientos
2. Diseño de experiencia UX/UI
3. Desarrollo técnico por sprints: infraestructura Firebase y login → lógica de sincronización y dashboard → integración con Gemini IA → refinamiento visual y gestión de errores
4. Validación y pruebas en dispositivos reales y emuladores

Ventajas de esta arquitectura

- Sin servidor propio: no es necesario arrancar ni mantener ningún servidor
 - Acceso universal: web y Android acceden a los mismos datos en tiempo real
 - Seguridad integrada: El authenticator y sus reglas de seguridad garantizan aislamiento
 - IA sin backend: la llamada a Gemini se realiza directamente desde el cliente
 - Resiliencia: si la IA falla, se generan hitos genéricos automáticamente
-

3.3.2 Base de datos — Cloud Firestore

Estructura de datos

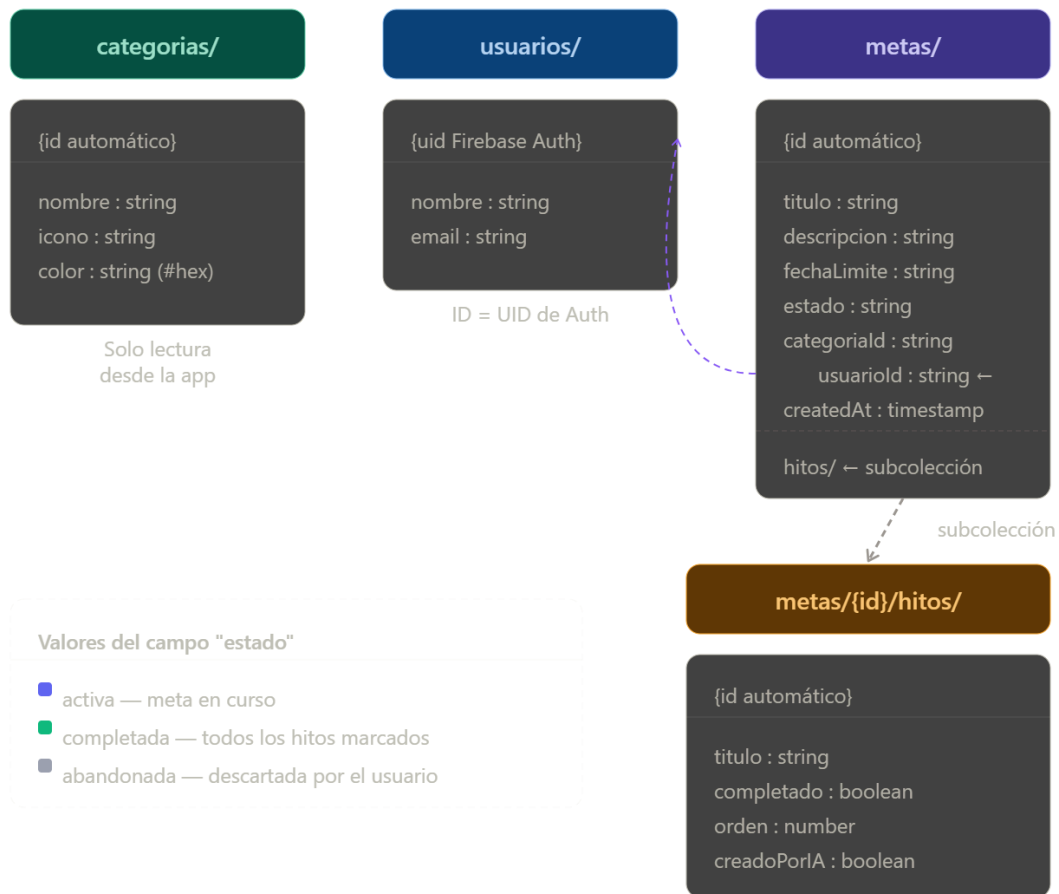


Figura 2. Estructura de datos de Firestore.

Nota: la app web guarda los hitos como array dentro del documento de meta. La app móvil (Samuel) los guarda como subcolección. Ambas soluciones son válidas y funcionan correctamente con Firestore.

Datos iniciales — categorías

Tabla 3. Categorías iniciales de la base de datos.

Nombre	Icono	Color
Salud	heart	#EF4444
Aprendizaje	book	#8B5CF6
Finanzas	money	#10B981
Personal	star	#F59E0B
Viajes	map	#3B82F6

Reglas de seguridad (Firestore Security Rules)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Categorías – solo lectura pública
    match /categorias/{id} {
      allow read: if true;
      allow write: if false;
    }

    // Metas – cada usuario solo ve las suyas
    match /metas/{metaId} {
      allow read, write: if request.auth != null
                                     && request.auth.uid ==
resource.data.userId;
      allow create: if request.auth != null;
```

```

// Hitos – subcolección de meta, acceso si está autenticado
match /hitos/{hitoId} {
  allow read, write: if request.auth != null;
}

// Usuarios – cada uno solo accede al suyo
match /usuarios/{userId} {
  allow read, write: if request.auth != null
    && request.auth.uid == userId;
}
}
}

```

Explicación actual de las reglas de Firestore

- **Categorías:** cualquier usuario, incluso sin iniciar sesión, puede leer las categorías. La escritura está completamente bloqueada, por lo que solo pueden modificarse desde la consola de Firebase o mediante el Admin SDK.
- **Metas — read/update/delete:** un usuario autenticado solo puede leer, actualizar o eliminar las metas cuyo campo `userId` coincida exactamente con su `uid` de autenticación.
- **Metas — create:** cualquier usuario autenticado puede crear metas. Actualmente no se valida en reglas que el campo `userId` del documento coincida con el usuario autenticado, por lo que esa validación depende del frontend.
- **Hitos (subcolección de metas):** cualquier usuario autenticado puede leer y escribir hitos de cualquier meta, independientemente del propietario de la meta padre. Las reglas actuales no verifican que el usuario sea dueño de la meta asociada.
- **Usuarios:** cada usuario autenticado solo puede leer y modificar su propio documento dentro de `usuarios/{userId}`, ya que el `userId` de la ruta debe coincidir con su `uid`.

Decisiones de diseño

1. Los hitos se guardan como subcolección de cada meta porque siempre se consultan en el contexto de una meta concreta.
 2. El campo `userId` permite identificar al propietario de cada meta y aplicar control de acceso directamente desde las reglas de Firestore.
 3. Las categorías son globales y de solo lectura desde el cliente.
 4. El UID de Firebase Authentication se utiliza como ID del documento del usuario en Firestore, relacionando autenticación y datos sin necesidad de claves adicionales.
 5. Los datos de un usuario eliminado no se eliminan automáticamente en cascada. Las metas pueden quedar almacenadas en Firestore sin propietario válido, aunque ya no serán accesibles para ningún usuario autenticado.
 6. El nombre del campo propietario (`userId`) debe mantenerse idéntico en todos los frontends y en las reglas de seguridad, ya que las reglas dependen directamente de ese campo para validar permisos.
 7. Actualmente, cualquier usuario autenticado puede crear hitos dentro de cualquier meta, ya que las reglas de la subcolección hitos solo verifican autenticación y no propiedad de la meta padre.
-

3.4. Lógica de datos e integración con IA

Cómo reemplaza Firebase al backend

Tabla 3: Comparativa entre Spring Boot y Firebase.

Antes (Spring Boot)	Ahora (Firebase + clientes)
MetaController + MetaService	addDoc / getDocs desde el cliente
HitoController + HitoService	Subcolección hitos gestionada desde el cliente
Spring Security + JWT	Firebase Authentication
PostgreSQL	Cloud Firestore
laService + OpenAI	Google AI Studio + Gemini 2.5 Flash

Flujo de creación de una meta con IA

- Usuario rellena el formulario (título, descripción, fecha, categoría)
- El cliente guarda la meta en Firestore → obtiene el metald
- El cliente llama a Gemini 2.5 Flash vía API REST con el prompt
- Gemini devuelve un array JSON con un número hitos personalizados
- Los hitos se limpian (comillas tipográficas → estándar) y se parsean
- Los hitos se guardan en la subcolección metas/{metald}/hitos
- El usuario ve los hitos generados y puede editarlos manualmente

Si la llamada a Gemini falla por cualquier motivo, el sistema genera automáticamente los hitos genéricos para que la app nunca se quede sin hitos.

Prompt utilizado

Eres un coach de productividad experto.

Divide este objetivo en exactamente 5 hitos concretos y accionables.

OBJETIVO: "{titulo}"

DESCRIPCIÓN: "{descripcion}"

FECHA LÍMITE: {fechaLimite}

IMPORTANTE: Responde SOLO con un array JSON. Sin texto antes ni después.

Formato: ["Hito uno", "Hito dos", "Hito tres", "Hito cuatro", "Hito cinco"]

Particularidades técnicas de la integración

- Modelo: gemini-2.5-flash
- Endpoint:
<https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent>
- maxOutputTokens: 2056 (mínimo necesario para evitar respuestas truncadas)
- Limpieza de comillas tipográficas (" ") antes del JSON.parse() para evitar errores de parseo cuando la IA incluye ejemplos entrecomillados:

```
const jsonLimpio = jsonMatch[0]
  .replace(/[\u201C\u201D]/g, '')
  .replace(/[\u2018\u2019]/g, '');
const hitos = JSON.parse(jsonLimpio);
```

Gestión de estado global — GoalsContext

Tanto la web como la app móvil centralizan la lógica en un contexto (GoalsContext) que adapta el patrón MVVM a React:

- **Modelo:** entidades de datos en Firestore (metas, usuarios)
- **Vista:** componentes de páginas y componentes UI
- **ViewModel:** GoalsProvider, que centraliza lógica de autenticación, operaciones CRUD e integración con IA

El contexto evita el "prop drilling" y garantiza una fuente única de verdad. El listener onSnapshot actualiza el estado de React en tiempo real ante cualquier cambio en Firestore, sin recargas manuales.

Operaciones CRUD en Firestore

Tabla 4: Operaciones CRUD en Firestore

Operación	Implementación
Listar metas	<code>query + where("usuarioid", uid) + onSnapshot</code>
Crear meta	<code>addDoc(collection(db, "metas"), meta)</code>
Actualizar meta	<code>updateDoc(doc(db, "metas", id), datos)</code>
Eliminar meta	<code>deleteDoc(doc(db, "metas", id))</code>
Listar hitos	<code>collection(db, "metas", id, "hitos") + orderBy("orden")</code>
Marcar hito	<code>updateDoc(hitoRef, { completado: !actual })</code>

4. Frontend Android

4.1 Análisis del estado del arte y elección tecnológica

4.1.1 Comparativa Crítica de Frameworks de Desarrollo Móvil

Para la realización de este proyecto, se consideraron y evaluaron tres alternativas principales bajo criterios de rendimiento, mantenibilidad, escalabilidad y tiempo de mercado

(Time-to-Market):

1. **Desarrollo Nativo (Kotlin para Android / Swift para iOS):** Aunque ofrece el máximo rendimiento teórico y acceso a las APIs más profundas del sistema, se descartó debido a que duplica el esfuerzo de desarrollo y mantenimiento. Para un proyecto individual o de equipo pequeño como un TFG, gestionar dos bases de código separadas es ineficiente y consume excesivos recursos temporales.
 2. **Flutter (Framework basado en Dart):** Se evaluó positivamente por su motor gráfico Skia y su consistencia visual en todas las plataformas. Sin embargo, su curva de aprendizaje para desarrolladores ya familiarizados con el ecosistema web (JavaScript/React) es más pronunciada, y su comunidad, aunque creciente, no iguala la ubicuidad y el soporte de JavaScript en la industria actual.
 3. **React Native (JavaScript/TypeScript):** Esta fue la opción ganadora por unanimidad. La capacidad de reutilizar el paradigma de React (declarativo, basado en componentes) permite un desarrollo ágil y modular. Sumado a la potencia del motor Hermes para la optimización de la ejecución de JavaScript y la facilidad de integrar librerías de terceros, lo convierte en la herramienta ideal. React Native permite una iteración rápida sin sacrificar la sensación de "app nativa" que el usuario final espera en términos de transiciones y respuesta táctil.
-

4.2 Arquitectura Técnica y Patrones de Diseño de Software

4.2.1 Adaptación del Patrón MVVM a React

Aunque el desarrollo móvil tradicional en Android (Kotlin/Java) suele utilizar fragmentos y ViewModels dedicados, en el desarrollo con React Native hemos adaptado el patrón **Model-View-ViewModel (MVVM)** para aprovechar la naturaleza reactiva y declarativa de los componentes funcionales:

- **Modelo (Model):** Los modelos son las entidades de datos que residen en la nube, específicamente en **Firebase Firestore**. En el código de la aplicación, esto se traduce en los objetos de datos que representan las "metas" y los "usuarios", conteniendo campos como el nombre, la descripción, la fecha límite, la categoría y el array de hitos.
- **Vista (View):** Son todos los componentes situados en los directorios `src/app/pages` y `src/app/components`. Estos componentes son "puros" o "tontos" en el sentido de que su única responsabilidad es recibir datos y funciones a través de "props" y renderizar la interfaz de usuario correspondiente. No conocen el origen de los datos ni la lógica compleja de persistencia. Esto facilita la reutilización y el mantenimiento del código.
- **ViewModel (Capa de Contexto):** El componente `GoalsProvider` actúa como nuestro ViewModel global. Es el encargado de centralizar la lógica de negocio, realizar las peticiones asíncronas a la API de Firebase, procesar las respuestas de la IA y exponer el estado y las funciones necesarias al resto de la aplicación a través de la Context API de React.

4.2.2 Gestión de Estado Global con Context API

Para evitar el fenómeno conocido como "Prop Drilling" (el paso de datos a través de múltiples niveles de componentes que no los necesitan), se ha implementado una arquitectura centralizada mediante el **Context API** de React:

- **Fuente Única de Verdad:** El contexto asegura que todos los componentes vean la misma información al mismo tiempo. Si una meta se completa en la pantalla de detalle, el Dashboard se actualiza instantáneamente sin necesidad de recargas manuales.
 - **Persistencia de Datos de Sesión:** El contexto se encarga de mantener al usuario autenticado y sus datos sincronizados mientras navega por las distintas pantallas de la aplicación, proporcionando una experiencia fluida y sin interrupciones.
 - **Desacoplamiento:** Permite separar completamente la lógica de acceso a datos de la representación visual, facilitando enormemente las pruebas unitarias y el mantenimiento del código a largo plazo.
-

4.3 Análisis Detallado de la Implementación por Ficheros (Lógica de Negocio)

4.3.1 El Corazón de la Lógica:

src/app/context/GoalsContext.jsx

Este fichero es, sin duda, el más importante de toda la aplicación. Define el GoalsProvider, que envuelve a toda la aplicación en App.jsx.

- **Importaciones Críticas:** Comienza importando las funciones de bajo nivel de Firebase, como auth para la seguridad y db (Firestore) para los datos. También importa funciones de React como createContext, useContext, useState y useEffect.
- **Estado del Usuario (Auth):** Define un estado user inicializado en null. Utiliza un useEffect para registrar un "listener" mediante onAuthStateChanged. Cuando Firebase detecta que un usuario ha iniciado sesión (ya sea por login directo o por sesión persistente almacenada localmente), recupera el nombre del usuario y su UID, actualizando el estado de la aplicación. Esto permite que la navegación cambie de la pantalla de Login al Dashboard de forma automática y fluida.
- **Sincronización de Datos (Firestore):** Implementa un segundo useEffect que depende del estado del usuario. Si hay un usuario logueado, crea una query a la colección "metas" donde el campo userId coincida con el del usuario actual. Usa onSnapshot para escuchar cambios. Cada vez que hay un cambio en la base de datos (un hito completado, una nueva meta añadida), Firestore envía los datos actualizados, el código los procesa, los ordena por fecha de creación (los más recientes primero) y actualiza el estado global goals.
- **Funciones de Interacción (API de la App):**
 - login(email, password): Realiza la llamada a la función de Firebase Auth correspondiente. Si tiene éxito, el listener de estado se encargará de actualizar el usuario en toda la app.
 - register(email, password, name): Crea el usuario en Firebase Auth y,

simultáneamente, crea un documento en la colección "usuarios" de Firestore para guardar el nombre de perfil.

- `addGoal(goal, initialMilestones)`: Esta función es clave para la integración con la IA. Toma los datos del formulario de creación y el array de textos devuelto por Gemini. Transforma esos textos en objetos estructurados con ID único (generado con `Date.now()`) y estado inicial `completed: false`, añade el `userId` y sube el documento completo a Firestore.
- `toggleMilestone(goalId, milestoneId)`: Busca la meta específica en el estado local, clona sus hitos para mantener la inmutabilidad de React, cambia el estado del hito seleccionado y actualiza el documento en la nube de forma asíncrona mediante `updateDoc`.

4.3.2 El Sistema de Rutas y Navegación: `src/app/routes.jsx`

Gestiona la navegación condicional y protegida de la aplicación.

- **Navegación basada en Estado Autenticado:** Utiliza el estado `user` del `GoalsContext`. Si el usuario es nulo (`null`), el navegador solo muestra la `LoginPage`. Si el usuario existe, el navegador cambia dinámicamente la pila de pantallas (`Stack`) para mostrar el `Dashboard` y la pantalla de `CreateGoal`.
 - **Seguridad por Diseño:** Al no incluir las pantallas privadas en el árbol de navegación cuando no hay un usuario autenticado, es técnicamente imposible para un usuario no autorizado acceder a las pantallas de datos, incluso intentando manipular la memoria del dispositivo o el historial de navegación.
-

4.4 Análisis Detallado de la Interfaz de Usuario (Capa de Presentación)

4.4.1 El Dashboard Central: `src/app/pages/DashboardPage.jsx`

Es la pantalla de mayor complejidad visual y funcional, donde el usuario pasa la mayor parte del tiempo.

- **Capa de Acceso a Datos:** Recupera las metas activas y el perfil del usuario mediante el hook personalizado `useGoals`.
- **Lógica de Negocio Local (Real-Time Stats):** Calcula en tiempo real las estadísticas que se ven en las tarjetas superiores. Realiza sumas de hitos y filtrados complejos para determinar el progreso global del usuario. Esto proporciona un feedback inmediato y gratificante que fomenta el uso continuado de la aplicación.
- **Componentes Modulares de UI:**
 - **StatCard:** Muestra las métricas críticas (Metas activas, Hitos completados, Metas logradas). Cada una tiene un icono y un color distintivo que facilita la lectura rápida.
 - **GoalListItem:** Es el componente que representa cada meta individual en la lista. Recibe el objeto meta y el porcentaje de progreso. Incluye lógica interna para calcular si la fecha límite está cerca y mostrar un aviso visual (badge) de color naranja (próximo) o rojo (vencido).
 - **Estado Vacío (Empty State):** Si el usuario no tiene metas registradas, en lugar de mostrar una lista vacía y triste, muestra una ilustración motivadora y un botón central para empezar. Esto mejora enormemente la retención de nuevos usuarios durante el proceso de "Onboarding".

4.4.2 Creación de Objetivos Asistida: `src/app/pages/CreateGoalPage.jsx`

Esta pantalla es el punto de entrada de la información del usuario hacia el motor de IA.

- **Formulario de Entrada Interactivo:** Gestiona estados locales para capturar el nombre, la descripción y la fecha de forma eficiente.
- **Selector de Categorías Temáticas:** Renderiza una lista de opciones con iconos visuales. Al seleccionar una categoría, el color de la meta se previsualiza instantáneamente en la pantalla, creando una conexión emocional y visual entre el usuario y su nuevo objetivo.
- **Integración de Fecha Nativa:** Utiliza un componente de calendario nativo del sistema operativo (Android/iOS) para asegurar que la fecha seleccionada sea válida y que el proceso de selección sea cómodo para el usuario.
- **Gestión de Carga y Feedback:** Mientras la IA de Gemini genera los hitos personalizados, el botón de "Crear" se deshabilita para evitar peticiones duplicadas y se muestra un spinner de carga con un mensaje informativo. Una vez recibidos los datos, guarda la meta en Firestore y redirige al Dashboard con una animación de éxito.

4.4.3. Detalle, Gestión e Interacción:

src/app/components/dashboard/GoalDetailView.jsx

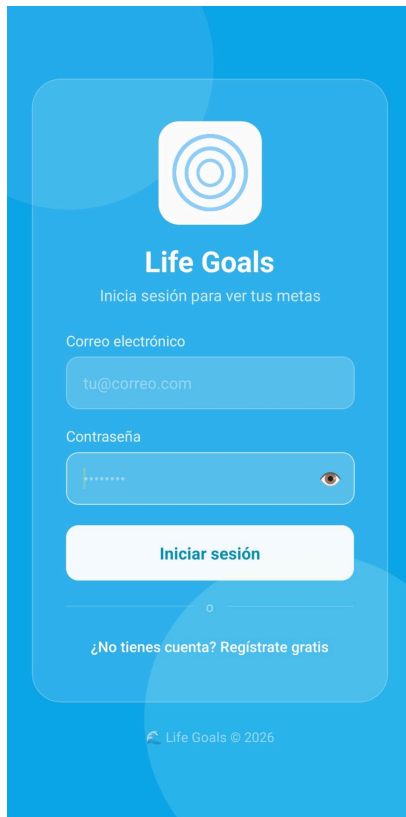
Esta es la vista donde ocurre la gestión diaria y detallada de los hitos.

- **Visualización de Progreso en tiempo real:** Muestra una barra de progreso grande, colorida y animada en la parte superior. A medida que el usuario marca hitos como completados, la barra se llena de forma fluida.
- **Lista de Hitos Interactiva y Flexible:** Cada hito es un componente interactivo. El usuario puede marcar tareas como completadas, borrar tareas que ya no considera necesarias o añadir nuevas tareas de forma manual para personalizar totalmente su camino al éxito.
- **Refuerzo Positivo y Gamificación:** Cuando todos los hitos se marcan como completados, la aplicación muestra un banner dorado de "Meta Lograda" con un trofeo brillante, aplicando principios básicos de gamificación para mantener la motivación y la satisfacción del usuario al completar sus retos.

4.5. Manual de Usuario Exhaustivo y Guía de Interacción Paso a Paso

4.5.1. Acceso Seguro y Gestión de su Cuenta Personal

1. **Registro de nueva cuenta:** Al abrir la aplicación por primera vez, pulse en el enlace "Regístrate gratis". Se le pedirá su nombre completo (que se usará para personalizar su experiencia), su correo electrónico y una contraseña segura de al menos 6 caracteres. Una vez completado el registro, accederá directamente a su panel personal de control.
2. **Inicio de sesión habitual:** Si ya tiene una cuenta creada, simplemente introduzca su email y contraseña en la pantalla de bienvenida. Si olvidó cerrar la sesión la última vez que usó la aplicación, LifeGoals le reconocerá automáticamente y le llevará directo al Dashboard sin pedirle los datos de nuevo.



[Figura 3. Login — Android]

3. **Privacidad y Seguridad:** Sus datos personales y sus metas se guardan de forma totalmente cifrada y segura en la infraestructura de Google. Solo usted tiene acceso a sus metas privadas mediante su contraseña personal, garantizando que sus sueños y planes estén protegidos.

4.5.2. Interpretar su Dashboard de Control Personal

El Dashboard es su resumen de vida y productividad. Está diseñado para darle toda la información necesaria de un solo vistazo.

- **Saludo Personalizado:** En la parte superior siempre verá un mensaje de bienvenida tipo "¡Hola, [Su Nombre]!".
- **Tarjetas de Métricas Rápidas:**
 - **Metas:** Le indica cuántos proyectos u objetivos tiene activos en este momento.
 - **Hitos:** Muestra su avance acumulado en todos sus proyectos (ej. "15/50 pasos dados en total"). Es un gran indicador de su esfuerzo diario.
 - **Logradas:** Es su vitrina de trofeos personal. Cuenta cuántas metas ha terminado satisfactoriamente al 100%.
- **Lista Dinámica de sus Metas:** Cada tarjeta de meta le da información vital de forma compacta: el nombre del objetivo, la categoría (mediante un icono visual), el porcentaje de avance actual y cuánto tiempo le queda para llegar a la fecha límite establecida.



[Figura 8. Dashboard — Android]

4.5.3. El Proceso de Creación de una Meta con Ayuda de la IA

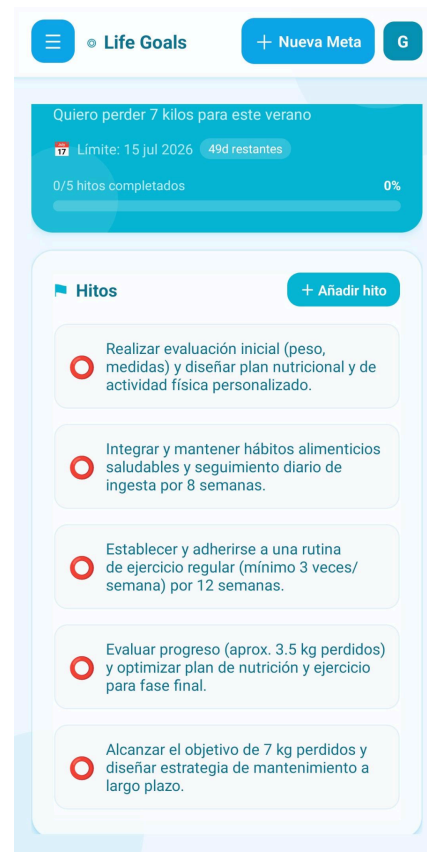
Crear un plan de acción nunca fue tan sencillo. Siga estos pasos:

1. En su Dashboard principal, pulse el botón azul con el signo **" + Nueva meta "**.
2. **Seleccione la Categoría:** Elija el icono que mejor represente su sueño u objetivo (Salud para ejercicio físico, Finanzas para ahorro económico, Aprendizaje para nuevos estudios, etc.).
3. **Nombre su Meta de forma clara:** Sea directo y positivo (ej. "Correr mi primera carrera popular de 10km").

4. **Describala con detalles:** Proporcione contexto a la IA para que pueda ser más precisa (ej. "Actualmente no realizo ejercicio físico de forma habitual y quiero empezar de cero siguiendo un plan progresivo").
5. **Establezca una Fecha Límite:** Pulse en el campo de fecha y elija el día en el que quiere haber cumplido su objetivo.
6. **Deje que la IA diseñe su plan:** Pulse en el botón ***** ✨ Crear Meta *****. Verá una animación de carga durante unos segundos. La IA de Google Gemini dividirá su gran objetivo en 5 pasos realistas, accionables y lógicos para usted, y los añadirá a su Dashboard automáticamente.



[Figura 9. Crear meta — Android]



[Figura 10. Detalle de meta — Android]

4.5.4. Gestión Diaria, Seguimiento y Cumplimiento

Para interactuar con una de sus metas, simplemente pulse sobre ella en la lista del Dashboard para entrar en su vista detallada.

- **Cumplir Pasos del Plan:** Cuando complete uno de los hitos del plan, simplemente pulse sobre el círculo o el texto del hito. Verá cómo el texto se tacha visualmente y su barra de progreso superior sube de forma inmediata. Es una forma muy gratificante de visualizar su avance constante.
 - **Personalizar su Hoja de Ruta:** Si un paso sugerido por la IA no le convence o no se adapta a su realidad, puede borrar ese hito pulsando el icono de la papelera que aparece a su derecha.
 - **Añadir sus Propios Pasos Manuales:** Si quiere ser más específico o añadir detalles que la IA no incluyó, use el campo de texto "Añadir hito" situado al final de la lista para escribir sus propios pasos de forma manual.
 - **Eliminar Metas Obsoletas:** Si por cualquier motivo decide que ya no quiere perseguir un objetivo determinado, puede usar el icono de la papelera en la esquina superior derecha de la pantalla de detalle. La aplicación le pedirá una confirmación de seguridad para estar seguro de que no borra nada por accidente.
-

4.6. Ciclo de Vida del Software y Metodología de Desarrollo Aplicada

El desarrollo de este proyecto se ha ejecutado siguiendo una metodología ágil e iterativa, permitiendo una adaptación rápida a los desafíos técnicos que surgieron durante la implementación de la IA. El proceso se dividió en cuatro etapas críticas perfectamente definidas:

1. **Fase de Análisis y Definición de Requerimientos:** Se identificó a través de investigación previa que la mayor causa de abandono de las metas personales es la falta de estructura y planificación inicial. Se decidió utilizar la potencia de los nuevos modelos de IA para resolver este "bloqueo de la página en blanco". Se eligió React Native como framework principal por su increíble velocidad de desarrollo y su capacidad multiplataforma.
2. **Fase de Diseño de Experiencia (UX/UI):** Se diseñó una interfaz limpia, minimalista y centrada en el contenido, basada en el uso de tarjetas, sombras suaves y barras de progreso claras. Se eligió una paleta de colores basada en distintos tonos de cian y azul para transmitir al usuario una sensación de orden, tecnología de vanguardia y tranquilidad mental. Se diseñaron los flujos de navegación para minimizar la fricción, asegurando que el usuario nunca tuviera que dar más de tres toques en pantalla para realizar cualquier acción importante.

3. Fase de Desarrollo Técnico por Sprints:

- **Sprint 1 (Infraestructura):** Montaje inicial de la infraestructura en la nube (Firebase Auth y Firestore Database). Implementación del flujo de Login.
- **Sprint 2 (Lógica Core):** Creación de la lógica central de sincronización de datos en tiempo real y desarrollo del Dashboard principal.
- **Sprint 3 (Inteligencia Artificial):** Integración profunda con la API de Google Gemini IA y desarrollo de la lógica de planificación automática de hitos.
- **Sprint 4 (UX/UI y Pulido):** Refinamiento visual final, implementación de animaciones de progreso fluidas y gestión robusta de errores de red.

4. Fase de Validación y Pruebas de Campo:

Se realizaron pruebas intensivas en múltiples dispositivos Android reales y emuladores con diferentes configuraciones de pantalla para asegurar que la aplicación es fluida, estable y que no consume recursos excesivos de batería o datos móviles en segundo plano.

4.7. Guía Técnica Completa para el Despliegue e Instalación (Para Desarrolladores)

4.7.1. Prerrequisitos de Software del Sistema

Para poder ejecutar, compilar o modificar este proyecto en su propia máquina de desarrollo, su ordenador debe contar obligatoriamente con los siguientes componentes instalados y configurados:

- **Node.js (v18.0 o superior):** El motor de ejecución fundamental para todo el entorno de desarrollo basado en React y JavaScript.
- **Java SDK (v17):** El kit de desarrollo de Java necesario para poder compilar y generar los binarios de la parte Android de la aplicación móvil.
- **Android Studio:** Correctamente instalado y configurado con el SDK de Android versión 34 (o compatible).
- **Variables de Entorno del Sistema:** Debe tener configurada y accesible la variable `ANDROID_HOME` en su sistema operativo para que las herramientas de React Native puedan encontrar el SDK.

4.7.2. Proceso de Instalación y Puesta en Marcha Paso a Paso

1. **Descarga del Código:** Clone el repositorio oficial o descargue el código fuente del proyecto en una carpeta local de su preferencia.
2. **Instalación de Dependencias:** Abra una terminal de comandos en la carpeta raíz del proyecto y ejecute el comando `npm install`. Esto descargará e instalará automáticamente todas las librerías y dependencias mencionadas en los apéndices de este documento.

3. Configuración del Backend en la Nube (Firebase):

- Debe crear un nuevo proyecto en la consola oficial de Firebase (<https://console.firebase.google.com/>).
- Descargue el archivo de configuración google-services.json desde la consola y colóquelo dentro de la carpeta android/app/ de su proyecto local.
- Active el método de autenticación por "Correo electrónico/Contraseña" en la sección de Authentication de la consola.
- Cree una base de datos de Firestore en "Modo de producción" y aplique las reglas de seguridad descritas anteriormente en la sección 7.2. * Copie las claves de configuración web de Firebase y péguelas en el archivo src/lib/firebase.js del proyecto.

4. **Configuración del Motor de IA:** Obtenga una clave de API gratuita para el modelo Gemini en Google AI Studio y péguela en el lugar correspondiente del archivo src/services/[iaService.js](#).

5. Ejecución de la Aplicación:

- Abra una terminal y ejecute: `npx react-native start` (esto iniciará el servidor de empaquetado Metro).
- Abra una segunda terminal y ejecute: `npx react-native run-android` (esto compilará la aplicación y la lanzará en un emulador abierto o en un dispositivo móvil conectado por USB).

5. Frontend Web

5.1. Marco teórico y tecnologías utilizadas

La aplicación combina tecnologías de **frontend moderno** con **servicios gestionados en la nube**. Esta combinación permite desarrollar una aplicación completa sin necesidad de crear un backend propio desde cero.

5.1.1 React y arquitectura basada en componentes

React es una biblioteca para construir interfaces de usuario mediante componentes reutilizables. Cada pantalla se implementa como un componente de página, mientras que los elementos repetibles se separan en componentes más pequeños:

- GoalListItem
- StatCard
- MilestoneItem
- CategorySelector

5.1.2 Vite como herramienta de desarrollo

Vite se utiliza como entorno de desarrollo y herramienta de construcción. Su configuración aparece en vite.config.ts, donde se incorporan los plugins de React y Tailwind y se define el alias @ hacia la carpeta src. Permite cargar variables de entorno con el prefijo VITE_.

5.1.3 TypeScript, rutas y estilos

- **TypeScript** → tipado estático para interfaces Goal, Milestone y User
- **React Router** → navegación con createBrowserRouter y rutas protegidas
- **Tailwind CSS** → tonos azules y turquesas, tarjetas translúcidas, bordes redondeados
- **motion/react** → animaciones ligeras

Tabla de tecnologías

Tabla 5. Tecnologías utilizadas en el frontend web.

Tecnología	Uso en el proyecto	Archivo relacionado
React	Construcción de pantallas y componentes	src/main.tsx, src/app/App.tsx
TypeScript	Tipado de datos, props, usuario, metas e hitos	GoalsContext.tsx y componentes
Vite	Servidor de desarrollo, build y variables de entorno	vite.config.ts, package.json
Firebase Auth	Registro, login, logout y escucha de sesión	src/lib/firebase.ts, GoalsContext.tsx
Cloud Firestore	Persistencia de usuarios, metas e hitos	Colecciones usuarios y metas
Gemini API	Generación automática de hitos para una meta	src/lib/gemini.ts
React Router	Navegación entre login, dashboard y creación	src/app/routes.tsx
Tailwind CSS / motion	Diseño responsive, estilos y animaciones	src/styles, componentes visuales

5.2. Desarrollo del proyecto

5.2.1 Arquitectura general

La arquitectura se basa en una **SPA** (Single Page Application) que cambia de vista mediante enrutado interno:

index.html

- └─ App.tsx (monta sobre #root)
- └─ RouterProvider (decide qué pantalla renderizar)
- └─ GoalsContext.tsx (capa de datos centralizada)

Tres bloques principales:

1. **Interfaz React** — pantallas y componentes
2. **Contexto de aplicación** — GoalsContext.tsx
3. **Servicios externos** — Firebase + Gemini

5.2.2 Estructura de carpetas y archivos

Tabla 6: Tecnologías utilizadas en el frontend web.

Archivo	Responsabilidad principal
src/main.tsx	Punto de entrada de React. Monta App sobre el div root
src/app/App.tsx	Envuelve la aplicación con GoalsProvider y RouterProvider
src/app/routes.tsx	Define rutas públicas, protegidas y redirecciones
src/app/context/GoalsContext.tsx	Gestiona usuario, metas, listeners, CRUD e integración con Gemini

Archivo	Responsabilidad principal
src/lib/firebase.ts	Inicializa Firebase App, Authentication y Firestore
src/lib/gemini.ts	Construye el prompt, llama a Gemini y devuelve hitos
src/app/pages/LoginPage.tsx	Pantalla de login y registro
src/app/pages/DashboardPage.tsx	Panel principal con métricas, listado y detalle de metas
src/app/pages/CreateGoalPage.tsx	Formulario de creación de meta con validación
src/app/components/Layout.tsx	Estructura superior, menú, botón de nueva meta y Outlet
src/app/components/Sidebar.tsx	Menú lateral con navegación, metas y cierre de sesión
src/styles/index.css	Carga fonts.css, tailwind.css y theme.css

5.2.3 Arranque de la aplicación

El ciclo de arranque comienza en main.tsx:

tsx

```
createRoot(document.getElementById("root")!).render(<App />);
```

App.tsx envuelve el router dentro de GoalsProvider. Esta decisión es importante porque las rutas protegidas necesitan consultar user y loading, valores que solo existen dentro del contexto.

5.2.4 Sistema de rutas y protección

Tabla 7: Estructura de carpetas y archivos del frontend web.

Ruta	Componente	Tipo de acceso
/	Redirección a /login	Inicial
/login	GuestRoute + LoginPage	Solo usuarios no autenticados
/dashboard	ProtectedLayout + DashboardPage	Solo usuarios autenticados
/create	ProtectedLayout + CreateGoalPage	Solo usuarios autenticados
*	Redirección a /login	Control de errores

Lógica de protección:

- ProtectedLayout → muestra spinner mientras Firebase comprueba la sesión. Si no hay usuario, redirige a /login
- GuestRoute → si el usuario ya está autenticado, lo redirige al dashboard

5.2.5 Autenticación de usuarios

La autenticación se gestiona desde GoalsContext.tsx mediante onAuthStateChanged, que queda suscrito al estado de sesión de Firebase.

Tabla 8: Sistema de rutas y protección.

Función	Descripción
login	Llama a signInWithEmailAndPassword, devuelve true/false
register	Crea usuario, actualiza displayName y guarda documento en colección usuarios
logout	Llama a signOut y limpia selectedGoalId

5.2.6 Gestión de datos con Firestore

Firestore almacena dos colecciones principales: usuarios y metas.

Campos de la colección metas:

Tabla 9: Funciones de autenticación y campos principales de metas.

Campo	Tipo	Descripción
nombre	string	Título de la meta
descripción	string	Explicación y contexto para Gemini
deadline	string	Fecha límite del formulario
category	string	Salud, Viajes, Finanzas, Personal o Aprendizaje

Campo	Tipo	Descripción
color	string	Color asociado a la categoría
milestones	array	Lista de hitos con id, text y completed
createdAt	string	Fecha de creación yyyy-mm-dd
userId	string	uid de Firebase del usuario propietario

5.2.7 GoalsContext como núcleo de la aplicación

GoalsContext.tsx define las interfaces Milestone, Goal y User, mantiene estados como user, goals, selectedGoalId y loading, y expone funciones para iniciar sesión, registrar usuario, cerrar sesión, añadir metas, borrar metas y modificar hitos.

Flujo completo de datos:

Usuario actúa en pantalla

- Pantalla llama a función del contexto
- Contexto modifica Firestore
- Firestore notifica cambio al listener (onSnapshot)
- Listener actualiza estado de React
- Pantallas se re-renderizan automáticamente

5.2.8 Integración de Gemini IA

La función `generateMilestones` en `src/lib/gemini.ts` recibe cuatro parámetros: título, descripción, categoría y fecha límite opcional.

Flujo interno de `generateMilestones`:

***Tabla 10:** Flujo interno de generación de hitos con Gemini.*

Paso	Descripción técnica
1	Comprobar si existe <code>VITE_GEMINI_API_KEY</code> . Si falta, devolver hitos genéricos
2	Construir un prompt con objetivo, descripción, categoría y fecha límite
3	Enviar petición POST al endpoint <code>generateContent</code> del modelo <code>gemini-2.5-flash</code>
4	Leer <code>data.candidates[0].content.parts[0].text</code>
5	Extraer el array JSON con una expresión regular
6	Parsear el JSON y validar que sea un array con contenido
7	Devolver los hitos generados o usar fallback en caso de error

```
tsx
```

```
const suggestedMilestones = await generateMilestones(
```

```
  goal.nombre,
```

```
  goal.descripcion,
```

```
  goal.category,
```

```
  goal.deadline
```

```
);
```

Si la API de Gemini no está disponible, la función devuelve una lista de hitos genéricos (hitosGenericos()), mejorando la robustez de la aplicación.

5.2.9 Pantallas de la aplicación

Tabla 11: Continuación del flujo de Gemini y pantallas principales

Pantalla / Vista	Archivo	Función principal
Login / Registro	LoginPage.tsx	Crear cuenta o iniciar sesión con email y contraseña
Dashboard	DashboardPage.tsx	Estadísticas, listado de metas, progreso y acciones
Crear meta	CreateGoalPage.tsx	Recoge datos de la meta, valida campos y llama a addGoal
Detalle de meta	GoalDetailView.tsx	Descripción, fecha límite, hitos, progreso y borrado

Pantalla / Vista	Archivo	Función principal
Menú lateral	Sidebar.tsx	Navegación, selección de metas y cierre de sesión

LoginPage — gestiona login y registro con un estado `isRegister`. Muestra errores sin exponer información técnica sensible.

DashboardPage — calcula estadísticas: número de metas, hitos totales, hitos completados y metas completadas. Si no hay metas, muestra un estado vacío con llamada a la acción.

CreateGoalPage — el usuario elige categoría, introduce nombre, descripción y fecha límite. La función `validate` comprueba los campos obligatorios. Si es correcto, llama a `addGoal` en `GoalsContext`.

5.2.10 Diseño UX/UI

La interfaz busca transmitir una sensación **ligera, positiva y motivadora**:

- Degradados azules y turquesas
- Fondos claros con tarjetas translúcidas
- Esquinas redondeadas
- Barras de progreso
- Animaciones con `motion/react` (entradas suaves, efectos `whileTap`, transiciones en modales)

5.2.11 Variables de entorno

El archivo `.env.example` enumera las variables necesarias para ejecutar la aplicación. Las credenciales reales no deben documentarse ni publicarse en la memoria. En el proyecto se usan variables con prefijo `VITE_`, lo que permite leerlas desde el frontend mediante `import.meta.env`. Este enfoque es cómodo para desarrollo, pero en producción debe tratarse con precaución porque las variables `VITE_` quedan incluidas en el bundle del cliente.

Tabla 12: Variables de entorno necesarias.

Variable	Uso
VITE_FIREBASE_API_KEY	Clave pública del proyecto Firebase
VITE_FIREBASE_AUTH_DOMAIN	Dominio de autenticación Firebase
VITE_FIREBASE_PROJECT_ID	Identificador del proyecto Firebase
VITE_FIREBASE_STORAGE_BUCKET	Bucket de Firebase Storage
VITE_FIREBASE_MESSAGING_SENDER_ID	Identificador de mensajería Firebase
VITE_FIREBASE_APP_ID	Identificador de la aplicación web en Firebase
VITE_GEMINI_API_KEY	Clave de acceso a la API de Gemini

Como mejora de seguridad, la llamada a Gemini podría moverse a una función backend o Cloud Function. De ese modo, la clave de Gemini no quedaría expuesta en el cliente y se podría controlar mejor el uso, la validación de entradas y los límites de petición. Para un TFG, la integración directa en frontend demuestra el flujo técnico, pero debe explicarse como una decisión de alcance y no como la solución más segura para un entorno productivo.

6. Resultados y pruebas realizadas

Plan de pruebas funcionales

Caso de prueba	Resultado esperado
Registro de usuario	Se crea el usuario en Firebase Auth y se guarda el perfil en usuarios/{uid}
Inicio de sesión	El usuario entra al dashboard y las rutas privadas quedan accesibles
Acceso sin sesión a /dashboard	La aplicación redirige a /login
Acceso con sesión a /login	La aplicación redirige a /dashboard
Crear meta con datos válidos	Se genera una lista de hitos, se guarda la meta y aparece en el dashboard
Crear meta con campos vacíos	El formulario muestra errores de validación y no guarda datos
Marcar hito como completado	El campo completed cambia y el porcentaje de progreso se actualiza
Añadir hito manual	Se agrega un nuevo objeto Milestone al array milestones

Caso de prueba	Resultado esperado
Eliminar meta	El documento de Firestore desaparece y se limpia la selección
Cierre de sesión	Se elimina la sesión activa y se vuelve al login

6.1 Resultados alcanzados

- Autenticación, registro y cierre de sesión conectados a Firebase
- Rutas privadas protegidas mediante estado global del usuario
- Creación de metas con validación, categorías, fecha límite y generación de hitos con Gemini
- Metas guardadas en Firestore y actualizadas en tiempo real mediante listeners
- Dashboard con métricas, progreso, listado, detalle y acciones sobre hitos
- Interfaz consistente y adaptada a uso móvil.

7. Conclusiones

7.1 Conclusiones del proyecto

El proyecto LifeGoals es una demostración práctica y sólida de cómo la Inteligencia Artificial de última generación puede integrarse de forma útil, ética y totalmente transparente en aplicaciones móviles cotidianas para mejorar la vida de las personas.

Así mismo el desarrollo del proyecto ha permitido aplicar de forma integrada varios contenidos del ciclo DAM: **diseño de interfaces, gestión de estado, consumo de servicios externos, autenticación, base de datos en la nube, rutas protegidas, validación de formularios y organización modular del código.**

7.2 Trabajo futuro

- Borrado en cascada de metas al eliminar usuario (Cloud Functions)
- Mover la llamada a Gemini a un backend seguro para no exponer la API Key en el cliente
- Sistema de notificaciones push inteligentes cuando se acerque el plazo de una meta
- Gamificación: niveles, puntos y medallas por hitos completados
- Sincronización con Google Calendar y calendarios nativos
- Modo oscuro adaptativo
- Internacionalización (EN, FR, DE)
- Conversación guiada con IA para refinar la meta antes de generar hitos
- Parte social: compartir metas y progreso con otros usuarios
- Versión PWA instalable o empaquetada para iOS

8. Bibliografía

- [1] React. *React Documentation*. Disponible en: <https://react.dev/>
 - [2] Vite. *Vite Documentation*. Disponible en: <https://vite.dev/>
 - [3] Firebase. *Firebase Authentication Documentation*. Disponible en: <https://firebase.google.com/docs/auth>
 - [4] Firebase. *Cloud Firestore Documentation*. Disponible en: <https://firebase.google.com/docs/firestore>
 - [5] Google AI for Developers. *Gemini API Documentation*. Disponible en: <https://ai.google.dev/gemini-api/docs>
 - [6] TypeScript. *The TypeScript Handbook*. Disponible en: <https://www.typescriptlang.org/docs/>
 - [7] React Router. *React Router Documentation*. Disponible en: <https://reactrouter.com/>
 - [8] Tailwind CSS. *Tailwind CSS Documentation*. Disponible en: <https://tailwindcss.com/docs>
 - [9] React Native. *React Native Documentation*. Disponible en: <https://reactnative.dev/docs/>
 - [10] React Navigation. *React Navigation Documentation*. Disponible en: <https://reactnavigation.org/docs/>
 - [11] MDN Web Docs. *JavaScript Reference*. Disponible en: <https://developer.mozilla.org/es/docs/Web/JavaScript>
-

Anexos

Anexo A: Estructura Completa y Pormenorizada del Árbol de Directorios (FrontEnd Android)

Para facilitar una auditoría técnica profunda del proyecto, a continuación se detalla la función específica de cada carpeta y subdirectorio principal del código fuente:

- `/android`: Contiene todo el código fuente nativo para la plataforma Android, incluyendo scripts de compilación Gradle, el Manifiesto de la aplicación, recursos visuales como el icono de la app y la lógica de inicio de la máquina virtual de Java.
- `/src/app/components`: Alberga las piezas visuales pequeñas, atómicas y reutilizables que forman la interfaz de usuario.
 - `/create-goal`: Contiene la lógica visual y los componentes específicos para el selector de iconos, colores y categorías de las metas.
 - `/dashboard`: Incluye los componentes especializados del panel de control, como las tarjetas de estadísticas globales y los items de la lista de metas.
 - `/ui`: Agrupa componentes base de diseño como botones genéricos, entradas de texto estilizadas, el logotipo de la aplicación o el componente Layout que sirve de envoltorio global para todas las páginas.
- `/src/app/context`: Es la ubicación del archivo `GoalsContext.jsx`, que actúa como el "almacén central" de datos y gestiona todo el estado global y la lógica de negocio de la aplicación mediante el uso de Hooks avanzados de React.
- `/src/app/pages`: Contiene los archivos de las tres pantallas principales y completas de la aplicación: la pantalla de acceso (Login), el panel de control (Dashboard) y la pantalla de creación de nuevos objetivos (CreateGoal).
- `/src/lib`: Contiene los archivos de configuración y conexión inicial con librerías externas de terceros, principalmente el SDK de Firebase.

- /src/services: Alberga los servicios de red y las funciones asíncronas encargadas de la integración directa con la API de Inteligencia Artificial de Google Gemini.
- /src/styles: Ubicación (si aplica en la arquitectura elegida) de las constantes de diseño, paletas de colores centralizadas y temas visuales de la aplicación.

Anexo B: Listado Detallado de Dependencias de Software Utilizadas (FrontEnd Android)

El proyecto LifeGoals se apoya de forma crítica en las siguientes librerías y paquetes de software libre de la comunidad JavaScript para garantizar su funcionamiento robusto y seguro:

- react-native (v0.85.3): El motor y framework principal que permite ejecutar código JavaScript como una aplicación nativa real en el dispositivo móvil.
- firebase (v12.13.0): El conjunto completo de herramientas y SDKs para gestionar la base de datos Firestore en tiempo real y la autenticación de usuarios.
- @react-navigation/native (v7.2.4): La librería estándar de la industria para gestionar de forma fluida el movimiento del usuario entre las distintas pantallas de la aplicación.
- @google/generative-ai (v0.24.1): El SDK oficial proporcionado por Google para realizar peticiones y conectar con la Inteligencia Artificial de Gemini de forma segura.
- @react-native-async-storage/async-storage (v1.23.1): La librería que permite guardar datos sencillos y persistentes en la memoria física del teléfono (como el token de sesión del usuario) de forma permanente.
- @react-native-community/datetimepicker (v9.1.0): El componente visual especializado que ofrece al usuario un selector de fechas nativo y cómodo para definir los plazos de sus metas.

Anexo C: — Árbol del proyecto (FrontEnd Web)src/

```
|— main.tsx
|— app/
|  |— App.tsx
|  |— routes.tsx
|  |— context/
|  |  └─ GoalsContext.tsx
|  |— pages/
|  |  |— LoginPage.tsx
|  |  |— DashboardPage.tsx
|  |  └─ CreateGoalPage.tsx
|  └─ components/
|     |— Layout.tsx
|     |— Sidebar.tsx
|     |— create-goal/
|     |  └─ CategorySelector.tsx
|     └─ dashboard/
|        |— GoalDetailView.tsx
|        |— GoalListItem.tsx
|        |— MilestoneItem.tsx
|        |— StatCard.tsx
|        └─ ConfirmDeleteModal.tsx
└─ lib/
   |— firebase.ts
   └─ gemini.ts
```

Anexo D: — Interfaces principales (FrontEnd Web)

ts

```
export interface Milestone {
```

```
  id: string;
```

```
  text: string;
```

```
  completed: boolean;
```

```
}
```

```
export interface Goal {
```

```
  id: string;
```

```
  nombre: string;
```

```
  descripcion: string;
```

```
  deadline: string;
```

```
  category: string;
```

```
  color: string;
```

```
  milestones: Milestone[];
```

```
  createdAt: string;
```

```
  userId: string;
```

```
}
```

Anexo E: — Variables de entorno de ejemplo (FrontEnd Web)

env

```
VITE_FIREBASE_API_KEY=your_api_key
```

```
VITE_FIREBASE_AUTH_DOMAIN=your_auth_domain
```

```
VITE_FIREBASE_PROJECT_ID=your_project_id
```

VITE_FIREBASE_STORAGE_BUCKET=your_storage_bucket

VITE_FIREBASE_MESSAGING_SENDER_ID=your_messaging_sender_id

VITE_FIREBASE_APP_ID=your_app_id

VITE_GEMINI_API_KEY=your_gemini_api_key

Anexo F: Glosario de Conceptos Técnicos Utilizados en el Proyecto

Para facilitar la comprensión a perfiles no técnicos, se definen a continuación los términos clave utilizados a lo largo de esta memoria:

- **API (Application Programming Interface):** Un conjunto de reglas, protocolos y herramientas que permiten que nuestra aplicación móvil "hable" y solicite información a la Inteligencia Artificial de Google en la nube de forma estructurada.
- **CRUD (Create, Read, Update, Delete):** Un acrónimo que describe las cuatro operaciones fundamentales y básicas que cualquier sistema de gestión de datos debe poder realizar (Crear, Leer, Actualizar y Borrar información).
- **Firestore:** El servicio de base de datos "en la nube" proporcionado por Firebase (Google) que destaca por no usar tablas rígidas, sino documentos flexibles en formato JSON, ideales para aplicaciones que necesitan escalar rápido.
- **Hook:** Una función especial introducida en las versiones modernas de React (como useState o useEffect) que permite a los desarrolladores manejar estados complejos y efectos secundarios dentro de componentes de función sencillos.

- **NoSQL:** Un tipo de arquitectura de base de datos moderna, flexible y no relacional que es ideal para aplicaciones móviles contemporáneas que necesitan evolucionar y cambiar su estructura de datos de forma rápida sin complicaciones técnicas.
- **Prompt:** La instrucción de texto natural (la orden) que le escribimos a la Inteligencia Artificial para indicarle exactamente qué resultado queremos obtener (en nuestro caso, la división de un objetivo en pasos lógicos).
- **Serverless:** Un modelo de desarrollo de software donde el programador no tiene que preocuparse por comprar, configurar o mantener servidores físicos en una oficina, sino que simplemente utiliza servicios inteligentes "en la nube" que se gestionan solos.